Smart Life: Smart Home Automation and Safety System Using ESP32

Yuvaraj Subramani.M[a], S Syed Mohammed Buhari[b], Mukesh Kumar.B[c]
Department of Artificial Intelligence & Machine learning (AIML)
Vels Institute of science , Technology & Advanced studies (VISTAS)
Pallavaram , Chennai – 600017
[a]*Email: 24619136@vistas.ac.in*
[b]*Email: 24619131@vistas.ac.in*
[c]*Email: 24619146@vistas.ac.in*

***Abstract*—**Smart Life is a self-hosted home automation dashboard that controls relays and infrared (IR) devices over Wi-Fi, with a built-in MQ-2 gas sensor safety system. An ESP32 DevKit hosts a lightweight web server that exposes REST-style endpoints for relay switching, PWM control, IR learning and replay, air-conditioner state, and gas readings. A browser-based dashboard, secured with hashed-password login, captcha, and OTP-based recovery, lets a user view and control every connected device from one screen using typed or voice commands. The system requires no cloud service or external hosting: the dashboard and the ESP32 firmware communicate directly over the same home Wi-Fi network. A dedicated safety routine continuously samples the MQ-2 sensor and, if a gas or smoke threshold is crossed, immediately triggers a relay-driven alarm, blinks an indicator LED, raises an on-screen alert with sound, and pushes a phone notification. The IR subsystem can learn the codes of any physical remote in a single button press, including automatic protocol detection for air conditioners, removing the need to look up manufacturer codes. Testing on a breadboard prototype with a 4-channel relay module, an IR receiver, and an MQ-2 sensor confirmed reliable relay switching, accurate remote-code learning and replay, and prompt safety-alert response. The result is a low-cost, self-hosted, and extensible smart-home platform suited for a single residence or a small research prototype.

Keywords*—ESP32; Home Automation; Internet of Things; Relay Control; IR Remote Learning; Gas Leak Detection; Voice Commands; Wi-Fi Dashboard.*

I. INTRODUCTION

The growth of low-cost Wi-Fi microcontrollers such as the ESP32 has made it practical for individual developers to build home-automation systems that once required dedicated hubs or subscription-based cloud platforms. Most consumer smart-home products, however, lock a user into a single brand's ecosystem, require an internet connection and a vendor cloud account even for local control, and rarely combine appliance switching with safety monitoring such as gas-leak detection in one place.

Smart Life addresses this gap with a self-hosted dashboard and an ESP32 firmware image that together control relays and IR-based appliances over the local Wi-Fi network, with no cloud dependency. The system also folds in an MQ-2 gas sensor so that a single low-cost board provides both convenience (lighting, fans, TVs, air conditioners) and a basic safety net (cooking-gas / smoke alerts). This paper describes the hardware, system architecture, dashboard design, IR-learning workflow, and safety logic of the Smart Life platform, and reports on a working breadboard prototype.

II. LITERATURE SURVEY

A number of ESP-based home automation systems have been proposed in prior work, generally focused on relay switching over Wi-Fi or Bluetooth from a mobile app or web page, with sensor-triggered alerts handled as a secondary feature [1], [2]. Several studies have looked specifically at MQ-series gas sensors for LPG and smoke detection, coupling a microcontroller-read analog threshold to a buzzer or relay-driven exhaust fan as the safety response [3]. IR-based universal remote replacement has also been explored using microcontroller IR receive/transmit libraries to record and replay existing remote codes rather than requiring a fixed code database, which avoids compatibility issues across appliance brands [4]. Voice or natural-language command interfaces for home automation dashboards have been studied as a way to reduce the friction of navigating multiple device screens, typically by mapping parsed commands onto the same REST endpoints used by the graphical interface [5]. Smart Life builds on these directions by combining local (non-cloud) Wi-Fi control, single-press IR learning with automatic air-

conditioner protocol detection, and an MQ-2-driven safety routine within one self-hosted dashboard, rather than treating them as separate add-ons.

III. SYSTEM ARCHITECTURE

The system has two communicating parts that sit on the same home Wi-Fi network, so no external server or cloud hosting is required. The Smart Life website provides the login, device dashboard, and IR-training screens. The ESP32 firmware runs a web server on port 80 that implements the relay, IR, and gas logic and exposes it through REST-style endpoints, listed in Table I.

Endpoint	Function
/relay?pin=N&state=1|0	Switch a relay channel
/pwm?pin=N&value=0-255	Brightness / speed control
/ir?code=NAME&state=1|0	Replay a learned remote button
/ir/learn?name=NAME	Teach a new IR button
/ir/ac?power=&temp=&mode=	Full air-conditioner state
/gas , /gas/threshold	Sensor reading and alert level
/status	All device states in one poll

TABLE I. ESP32 REST ENDPOINTS

IV. HARDWARE COMPONENTS

- **Controller:** ESP32 DevKit — runs the web server, Wi-Fi, IR, and gas logic
- **Relay module:** 4-channel relay switches the light and fan; two channels are spare
- **Power:** LM2596 buck converter steps the supply down to a clean 5V for the relay board
- **Gas sensor:** MQ-2 detects cooking-gas / smoke through an analog ADC1 line
- **IR receiver:** Learns codes directly from a real remote
- **Indicator:** LED and resistor give a visual gas-alert signal

V. SOFTWARE AND DASHBOARD DESIGN

A. Access and Login

- Username and SHA-256 hashed password stored permanently in the browser, with no forced expiry
- Captcha shown on both sign-in and sign-up
- Forgot-password flow with a 6-digit OTP step
- “Keep me signed in” controls whether the session survives closing the browser tab

B. Dashboard

The dashboard lists every connected device as a card — lights and fans on relay channels, and TVs or air conditioners on IR control — with live on/off, brightness, fan-speed, or temperature state. A single command bar accepts typed or voice commands such as “turn on hall AC”, “turn off all”, or “status”, which are parsed and mapped onto the same REST endpoints used by the on-screen controls (Fig. 1).

Fig. 1. Smart Life dashboard — device cards and command bar

VI. IR REMOTE LEARNING

- Add a TV or AC as a gadget, then flip its card into Learn mode
- Point the real remote at the ESP32's IR receiver and press a button once
- The ESP32 decodes it with the IRremoteESP8266 library and stores it against that button
- Turning Learn mode off makes the same dashboard button replay the code
- Air-conditioner protocol can be auto-detected from a single remote press instead of being picked manually

VII. SAFETY SYSTEM

The gas-safety routine runs continuously and follows a fixed four-step response once the MQ-2 reading crosses the configured alert threshold, shown in Fig. 2:

- **1)** MQ-2 reading crosses the threshold
- **2)** ESP32 turns the safety relay ON and blinks its indicator LED — instantly, with no dashboard open
- **3)** The dashboard shows a red alert banner and plays an alert tone
- **4)** A phone notification fires, either through the browser or a Telegram bot that needs no separate hosting

VIII. CONFIGURATION

- ESP32 IP address is the only setting that changes per Wi-Fi network
- Gas-alert threshold is tunable from the dashboard and saved to the board's flash memory
- Devices and the ESP32 IP address are saved per account, so they persist between sessions

IX. RESULTS

The system was built and tested on a breadboard prototype (Fig. 3) using an ESP32 DevKit, a 4-channel relay module, an MQ-2 gas sensor, an IR receiver, and an LM2596 buck converter. Relay switching and PWM commands issued from the

dashboard were reflected on the hardware with no perceptible delay over the local Wi-Fi network. IR codes learned from real TV and air-conditioner remotes replayed correctly from the dashboard, and the automatic protocol-detection feature correctly identified the air-conditioner's IR protocol from a single learning press. Raising the MQ-2 reading above the configured threshold reliably triggered the full safety sequence — relay-driven alarm, blinking LED, red dashboard banner with sound, and a phone notification — within the sensor's response time.

Fig. 3. Breadboard prototype — ESP32, relay module, and MQ-2 gas sensor

X. CONCLUSION AND FUTURE SCOPE

Smart Life demonstrates that a single ESP32 board, combined with a self-hosted dashboard, can provide unified relay and IR appliance control, voice/typed commands, and MQ-2-based gas safety monitoring without any cloud dependency. The breadboard prototype confirmed reliable device control, single-press IR learning, and prompt safety-alert response. Planned improvements include moving the dashboard off a local file and onto a small always-on server, such as a Raspberry Pi or a low-cost VPS, so the system remains reachable from outside the home network; adding a MOSFET or transistor driver stage for true PWM dimming of lights and fan speed rather than on/off-only control; adding a service worker so that phone alerts continue to arrive even when the dashboard site is fully closed; and adding further sensors, such as temperature, humidity, or a dedicated smoke detector, alongside the MQ-2 for a fuller safety picture.

REFERENCES

[1] R. Piyare and M. Tazil, “Bluetooth based home automation system using cell phone,” in Proc. IEEE 15th Int. Symp. Consumer Electronics (ISCE), 2011, pp. 192–195.

[2] K. Sonar, S. Baviskar, and V. Panchal, “IoT based home automation system over cloud,” Int. J. Comput. Appl., vol. 170, no. 5, pp. 1–5, 2017.

[3] S. Saha, A. Majumdar, and S. Pal, “Gas leakage detection and alert system using IoT,” Int. J. Eng. Res. Technol., vol. 6, no. 5, pp. 210–213, 2018.

[4] A. Kumar and R. Verma, “Universal IR remote controller using microcontroller,” Int. J. Comput. Sci. Eng., vol. 5, no. 3, pp. 45–49, 2017.

[5] P. Sharma and N. Gupta, “Voice controlled home automation system using IoT,” Int. J. Sci. Res. Comput. Sci. Eng., vol. 6, no. 4, pp. 12–16, 2018.